



هوش مصنوعی

بهار ۱۴۰۵

استاد: ماری اوریاد
گردآورندگان: رادین جریره، فاطمه جعفرزاده،
معین حسینی، آریانا زال‌نژاد، علی نوری

مهلت ارسال: ۳۰ خرداد

یادگیری ماشین

تمرین چهارم

- مهلت ارسال پاسخ تا ساعت ۲۳:۵۹ روز مشخص شده است.
- در طول ترم امکان ارسال با تاخیر پاسخ همه‌ی تمارین تا سقف ۴ روز و در مجموع ۱۰ روز، وجود دارد. پس از گذشت این مدت، پاسخ‌های ارسال‌شده پذیرفته نخواهند بود. همچنین، به ازای هر ساعت تأخیر غیر مجاز ۰.۵ درصد از نمره تمرین به صورت ساعتی کسر خواهد شد.
- هم‌کاری و هم‌فکری شما در انجام تمرین مانعی ندارد اما پاسخ‌های ارسال‌شده هر کس حتماً باید توسط خود او نوشته شده باشد.
- در صورت هم‌فکری و یا استفاده از هر منابع خارج درسی، نام هم‌فکران و آدرس منابع مورد استفاده برای حل سوال مورد نظر را ذکر کنید.
- لطفاً تصویری واضح از پاسخ سوالات نظری بارگذاری کنید. در غیر این صورت پاسخ شما تصحیح نخواهد شد.
- پاسخنامه سوالات را در سامانه کوئرا و به فرمت `AI_HW4_<STUDENT_ID>` بارگزاری کنید.
- نمره امتیازی بخش تئوری این تمرین تنها روی بخش تئوری همین تمرین اعمال شده و بر روی باقی بخش‌های درس **سوریز ندارد**.

سوالات نظری (۱۰۰+۱۰ نمره)

۱. (۱۶ نمره) با ذکر دلیل، درست و غلط بودن گزاره‌های زیر را مشخص نمایید.
 - (آ) در الگوریتم Naive Bayes فرض می‌شود که ویژگی‌های مسئله (Attributes یا Features) نسبت به یک دیگر مستقل هستند.
 - (ب) هدف اصلی از Smoothing در Naive Bayes جلوگیری از صفر شدن احتمال Posterior و وقوع Overfitting است.
 - (ج) درخت تصمیم تنها برای مسائل دسته‌بندی (Classification) قابل استفاده است.
 - (د) به طور کلی هر چه عمق درخت تصمیم بیشتر باشد، احتمال Overfitting بیشتر است.
 - (ه) اگر یک ویژگی تعداد بسیار زیادی مقدار یکتا داشته باشد، معیار Information Gain تمایل به انتخاب آن دارد.
 - (و) درخت تصمیم نمی‌تواند روابط غیرخطی بین ویژگی‌ها را مدل کند.
 - (ز) اگر عملکرد مدل روی دادگان Train و Validation ضعیف باشد، مشکل غالباً Underfitting است.
 - (ح) برای یافتن مقادیر مناسب ابرپارامترها (Hyper-parameters) از مجموعه دادگان آموزش استفاده می‌شود.
- حل.**

(آ) **نادرست.** فرضی که در Naive Bayes برقرار می‌شود، استقلال **شرطی** ویژگی‌ها به شرط دانستن کلاس است، نه استقلال مطلق (حاشیه‌ای) آن‌ها نسبت به یک‌دیگر. یعنی فرض می‌شود

$$\mathbb{P}(X_1, X_2, \dots, X_n | Y) = \prod_{i=1}^n \mathbb{P}(X_i | Y),$$

که با $\mathbb{P}(X_i, X_j) = \mathbb{P}(X_i)\mathbb{P}(X_j)$ (استقلال مطلق) یکی نیست. دو ویژگی می‌توانند به‌طور کلی وابسته باشند اما به شرط کلاس مستقل شوند؛ بنابراین گزاره به‌شکل نوشته‌شده دقیق نیست.

(ب) **درست.** وقتی یک مقدار از یک ویژگی برای یک کلاس در دادگان آموزش دیده نشده باشد، برآورد بیشینه درست‌نمایی برای احتمال شرطی $\mathbb{P}(X_i | Y)$ برابر صفر می‌شود و چون این احتمال‌ها در هم ضرب می‌شوند، کل احتمال Posterior صفر خواهد شد (مسئله‌ی فراوانی صفر). هموارسازی (Smoothing) مانند Laplace با افزودن یک شمارنده‌ی مصنوعی از صفر شدن این احتمال‌ها جلوگیری می‌کند و هم‌زمان نقش یک تنظیم‌کننده (Regularization) را ایفا کرده و از اتکای بیش از حد مدل به دادگان آموزش (و در نتیجه Overfitting) می‌کاهد.

(ج) **نادرست.** درخت تصمیم علاوه بر دسته‌بندی برای مسائل رگرسیون (Regression) نیز به کار می‌رود. در درخت رگرسیون، به جای برچسب کلاس، در هر برگ یک مقدار پیوسته (مثلاً میانگین نمونه‌های آن برگ) پیش‌بینی می‌شود و معیار انشعاب نیز به جای آنتروپی/جینی، معیارهایی مانند کاهش واریانس یا MSE است.

(د) **درست.** هر چه عمق درخت بیشتر شود، تعداد برگ‌ها و انعطاف مدل بیشتر شده و درخت می‌تواند مرزهای پیچیده‌تری بسازد؛ در نتیجه به جای یادگیری الگوی کلی داده، نویز و جزئیات دادگان آموزش را نیز حفظ می‌کند. این افزایش Variance احتمال Overfitting را بالا می‌برد.

(ه) **درست.** معیار Information Gain به ویژگی‌هایی با تعداد مقادیر یکتای زیاد (بزرگ بودن Cardinality) گرایش دارد. چنین ویژگی‌ای داده را به گروه‌های بسیار کوچک و Pure تقسیم می‌کند و آنتروپی فرزندان را تقریباً صفر می‌سازد، پس Information Gain آن به‌طور مصنوعی بزرگ می‌شود.

(و) **نادرست.** درخت تصمیم ذاتاً یک مدل غیرخطی است. اگرچه هر انشعاب یک برش موازی محورها است، اما ترکیب سلسله‌مراتبی این برش‌ها می‌تواند مرزهای تصمیم غیرخطی و پیچیده و روابط غیرخطی میان ویژگی‌ها را مدل کند.

(ز) **درست.** ضعف هم‌زمان روی دادگان Train و Validation نشانه‌ی Bias بالا و ناتوانی مدل در یادگیری الگوی داده (حتی روی داده‌ای که آن را دیده) است که همان Underfitting است. در مقابل، Overfitting با عملکرد خوب روی Train و ضعیف روی Validation مشخص می‌شود.

(ح) **نادرست.** برای تنظیم ابرپارامترها (Hyper-parameters) از مجموعه‌ی اعتبارسنجی (Validation Set) استفاده می‌شود و نباید از خود مجموعه آموزش برای آن استفاده کرد. نگاه دیگر این است که Validation Set یک بخش جدا شده از خود دادگان آموزش است و در صورت در نظر گرفتن این فرض گزاره این سوال درست خواهد بود.

۲. (۱۳ نمره) شما به عنوان یک مهندس یادگیری ماشین در یک شرکت فناوری، مسئول طراحی یک سیستم تشخیص ایمیل‌های اسپم هستید. داده‌های آموزشی شما شامل ۱۰ ایمیل با ویژگی‌های زیر است:

- X_1 : آیا فرستنده در لیست مخاطبین است؟ (۱: بله، ۰: خیر)
- X_2 : آیا ایمیل حاوی لینک‌های مشکوک است؟ (۱: بله، ۰: خیر)
- X_3 : آیا در متن ایمیل از کلمات پرخاشگرانه استفاده شده است؟ (۱: بله، ۰: خیر)
- کلاس: ایمیل اسپم است (۱) یا خیر (۰).

(آ) محاسبات لازم برای انتخاب بهترین ویژگی جهت انشعاب در گره ریشه را انجام دهید.

(ب) تیم مدیریت شرکت اصرار دارد که مدل باید دقت ۱۰۰٪ روی داده‌های آموزشی داشته باشد، بنابراین پیشنهاد می‌دهند درخت را تا جایی رشد دهیم که تمامی گره‌های برگ Pure شوند. آیا در این صورت مدل لزوماً برای داده‌های تست بهتر عمل می‌کند؟ پاسخ خود را با ذکر دلیل توضیح دهید.

Class	X_3	X_2	X_1	ID
۰	۱	۱	۰	۱
۱	۰	۱	۱	۲
۰	۰	۰	۱	۳
۱	۱	۱	۱	۴
۱	۱	۱	۰	۵
۰	۱	۰	۱	۶
۰	۰	۱	۰	۷
۰	۱	۱	۰	۸
۰	۱	۰	۱	۹
۰	۰	۱	۱	۱۰

(ج) علاوه بر معیار Information Gain، یک معیار دیگر برای انتخاب فیچر در درخت تصمیم به نام Gini Impurity داریم. درمورد چگونگی استفاده از این معیار برای انتخاب ویژگی انشعاب درخت تحقیق کنید و مجددا محاسبات لازم برای انتخاب بهترین ویژگی جهت انشعاب در گره ریشه را با استفاده از این معیار انجام دهید.

حل.

(آ) ابتدا توزیع کلاس در گره ریشه را می‌یابیم. از میان ۱۰ ایمیل، ۳ ایمیل اسپم (Class = 1) و ۷ ایمیل غیراسپم (Class = 0) هستند. آنتروپی گره ریشه برابر است با

$$H(S) = -\frac{3}{10} \log_2 \frac{3}{10} - \frac{7}{10} \log_2 \frac{7}{10} = 0.881 \text{ bit}.$$

اکنون برای هر ویژگی، پس از انشعاب، آنتروپی وزنی فرزندان و سپس Information Gain را محاسبه می‌کنیم:

$$IG(S, X) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v).$$

ویژگی X_1 (حضور در لیست مخاطبین):

- $X_1 = 1$: نمونه ۶ (IDهای ۲، ۳، ۴، ۶، ۹، ۱۰) شامل ۲ اسپم و ۴ غیراسپم.
- $X_1 = 0$: نمونه ۴ (IDهای ۱، ۵، ۷، ۸) شامل ۱ اسپم و ۳ غیراسپم.

$$H(X_1=1) = -\frac{2}{6} \log_2 \frac{2}{6} - \frac{4}{6} \log_2 \frac{4}{6} = 0.918, \quad H(X_1=0) = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} = 0.811.$$

$$IG(S, X_1) = 0.881 - \left(\frac{6}{10} (0.918) + \frac{4}{10} (0.811) \right) = 0.881 - 0.875 = 0.006.$$

ویژگی X_2 (وجود لینک مشکوک):

- $X_2 = 1$: نمونه ۷ (IDهای ۱، ۲، ۴، ۵، ۷، ۸، ۱۰) شامل ۳ اسپم و ۴ غیراسپم.
- $X_2 = 0$: نمونه ۳ (IDهای ۳، ۶، ۹) شامل ۰ اسپم و ۳ غیراسپم؛ این گره Pure است.

$$H(X_2=1) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.985, \quad H(X_2=0) = 0.$$

$$IG(S, X_2) = 0.881 - \left(\frac{7}{10} (0.985) + \frac{3}{10} (0) \right) = 0.881 - 0.690 = 0.192.$$

ویژگی X_3 (کلمات پرخاشگرانه):

- $X_3 = 1$: نمونه ۶ (IDهای ۱، ۴، ۵، ۶، ۸، ۹) شامل ۲ اسپم و ۴ غیراسپم.

• $X_3 = 0$: نمونه ۴ (IDهای ۲، ۳، ۷، ۱۰) شامل ۱ اسپم و ۳ غیراسپم.

توزیع فرزندان دقیقاً مانند X_1 است، پس

$$IG(S, X_3) = 0.881 - \left(\frac{6}{11} (0.918) + \frac{4}{11} (0.811) \right) = 0.006.$$

نتیجه:

$$IG(X_2) = 0.192 > IG(X_1) = IG(X_3) = 0.006.$$

بنابراین ویژگی X_2 (وجود لینک مشکوک) بیشترین Information Gain را دارد و به عنوان بهترین ویژگی برای انشعاب در گره ریشه انتخاب می شود. شهوداً نیز منطقی است، چون این ویژگی به تنهایی یک شاخه ی کاملاً Pure (غیراسپم) تولید می کند.

(ب) **خیر، لزوماً بهتر عمل نمی کند.** رشد دادن درخت تا زمانی که تمام برگ ها Pure شوند تضمین می کند که دقت روی دادگان آموزش به ۱۰۰٪ برسد، اما این به معنای عملکرد بهتر روی دادگان تست نیست. چنین درختی معمولاً بسیار عمیق و پیچیده می شود و به جای یادگیری الگوی کلی مسئله، نویز و جزئیات و حتی نمونه های پرت (Outliers) موجود در دادگان آموزش را نیز حفظ می کند. این پدیده همان Overfitting است: مدل Variance بسیار بالایی پیدا کرده و توان تعمیم (Generalization) خود را از دست می دهد، پس روی داده های دیده نشده ی تست ضعیف عمل می کند.

برای مثال در همین داده، ممکن است یک برگ تنها بر اساس یک ایمیل خاص ساخته شود که تصادفاً یک ویژگی نادر دارد؛ چنین قاعده ای روی تست قابل اتکا نیست. راهکار درست، کنترل پیچیدگی درخت است: محدود کردن عمق (Max Depth)، تعیین حداقل تعداد نمونه در هر برگ، یا هرس کردن (Pruning) درخت، و انتخاب این ابرپارامترها بر اساس عملکرد روی مجموعه ی Validation. هدف نهایی، کمینه کردن خطای تعمیم است، نه خطای آموزش.

(ج) در معیار Gini Impurity، به جای آنتروپی از ناخالصی جینی برای سنجش عدم خلوص یک گره استفاده می شود:

$$Gini(S) = 1 - \sum_i p_i^2,$$

که در آن p_i نسبت نمونه های کلاس i در آن گره است. برای انتخاب ویژگی انشعاب، پس از هر انشعاب ناخالصی جینی وزنی فرزندان را حساب می کنیم و ویژگی ای را برمیگزینیم که این مقدار را **کمینه** کند (معادل بیشینه کردن کاهش جینی):

$$Gini_{split}(S, X) = \sum_v \frac{|S_v|}{|S|} Gini(S_v).$$

ناخالصی جینی گره ریشه:

$$Gini(S) = 1 - \left(\left(\frac{3}{11} \right)^2 + \left(\frac{8}{11} \right)^2 \right) = 1 - 0.58 = 0.42.$$

ویژگی X_1 : فرزندان (1, 3) و (2, 4):

$$Gini(X_1=1) = 1 - \left(\left(\frac{2}{6} \right)^2 + \left(\frac{4}{6} \right)^2 \right) = 0.444, \quad Gini(X_1=0) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = 0.375,$$

$$Gini_{split}(X_1) = \frac{6}{11} (0.444) + \frac{4}{11} (0.375) = 0.417.$$

ویژگی X_2 : فرزندان (0, 3) و (3, 4):

$$Gini(X_2=1) = 1 - \left(\left(\frac{2}{5} \right)^2 + \left(\frac{4}{5} \right)^2 \right) = 0.48, \quad Gini(X_2=0) = 0,$$

$$Gini_{split}(X_2) = \frac{5}{11}(\frac{1}{490}) + \frac{3}{11}(0) = 0.343.$$

ویژگی X_3 : توزیع فرزندان مانند X_1 است، پس

$$Gini_{split}(X_3) = 0.417.$$

نتیجه:

$$Gini_{split}(X_2) = 0.343 < Gini_{split}(X_1) = Gini_{split}(X_3) = 0.417.$$

کمترین ناخالصی جینی وزنی مربوط به ویژگی X_2 است، بنابراین طبق معیار Gini Impurity نیز ویژگی X_2 (وجود لینک مشکوک) به عنوان بهترین ویژگی برای انشعاب در گره ریشه انتخاب می شود؛ این نتیجه با انتخاب مبتنی بر Information Gain سازگار است.

۳. (۱۵ نمره) فرض کنید هر ایمیل شامل K کلمه به صورت $w_1, \dots, w_K$ و یک برچسب دودویی Y است که مقدار spam یا ham می گیرد. قاعده تصمیم گیری در مدل Naïve Bayes که از فرض Bag-of-Words بودن ایمیل استفاده می کند به صورت زیر خواهد بود:

$$\arg \max_y P(Y = y \mid w_1, \dots, w_K) = \arg \max_y P(Y = y) \prod_{i=1}^K P(W = w_i \mid Y = y)$$

(آ) فرض کنید مدل خود را بر روی مجموعه بزرگی از ایمیل ها آموزش داده اید و به احتمالات زیر رسیده اید:

W	best	today	price
$P(W \mid Y = \text{spam})$	$\frac{1}{4}$	$\frac{1}{6}$	$\frac{1}{4}$
$P(W \mid Y = \text{ham})$	$\frac{1}{12}$	$\frac{1}{3}$	$\frac{1}{6}$

ایمیل "best price" به شما داده می شود. احتمال $P(Y = \text{spam})$ در توزیع پیشین $P(Y)$ باید از چه مقداری بزرگتر باشد تا این ایمیل به عنوان spam طبقه بندی شود؟

(ب) حالا فرض کنید که می خواهید مدل جدیدی را آموزش بدهید و امکان استفاده از مدل بخش قبل را ندارید. پنج ایمیل زیر به عنوان داده ی آموزشی در اختیار شما قرار گرفته است:

- special offer just today (Spam)
- summer season off just for you (Spam)
- meeting today (Ham)
- call me later (Ham)
- see you tomorrow (Ham)

بدون استفاده از smoothing، مقادیر زیر را محاسبه کنید:

- $P(W = \text{just} \mid Y = \text{spam})$
- $P(W = \text{call} \mid Y = \text{ham})$
- $P(W = \text{offer} \mid Y = \text{ham})$
- $P(Y = \text{spam})$

حل.

(آ) ایمیل ”best price” تنها شامل دو کلمه‌ی best و price است. طبق قاعده‌ی Naïve Bayes، این ایمیل زمانی spam طبقه‌بندی می‌شود که امتیاز کلاس spam از امتیاز کلاس ham بزرگ‌تر باشد:

$$\mathbb{P}(Y = \text{spam}) \mathbb{P}(\text{best} | \text{spam}) \mathbb{P}(\text{price} | \text{spam}) > \mathbb{P}(Y = \text{ham}) \mathbb{P}(\text{best} | \text{ham}) \mathbb{P}(\text{price} | \text{ham}).$$

مقدار پیشین را $p = \mathbb{P}(Y = \text{spam})$ و در نتیجه $\mathbb{P}(Y = \text{ham}) = 1 - p$ می‌گیریم. با جای‌گذاری احتمال‌های شرطی داده‌شده:

$$p \cdot \frac{1}{8} \cdot \frac{1}{4} > (1 - p) \cdot \frac{1}{12} \cdot \frac{1}{6},$$

$$\frac{p}{32} > \frac{1 - p}{72}.$$

با ضرب طرفین در 32×72 و مرتب کردن:

$$72p > 32(1 - p) \implies 104p > 32 \implies p > \frac{32}{104} = \frac{4}{13} \approx 0.308.$$

بنابراین احتمال پیشین $\mathbb{P}(Y = \text{spam})$ باید از $\frac{4}{13} \approx 0.308$ بزرگ‌تر باشد تا این ایمیل به عنوان spam طبقه‌بندی شود.

(ب) برآورد بیشینه درست‌نمایی (MLE) احتمال شرطی هر کلمه، نسبت شمارش آن کلمه در همه‌ی ایمیل‌های یک کلاس به تعداد کل کلمات (توکن‌های) آن کلاس است، و احتمال پیشین هر کلاس نسبت تعداد ایمیل‌های آن کلاس به کل ایمیل‌هاست. نخست تعداد کل توکن‌های هر کلاس را می‌شماریم:

- Spam: “special offer just today” (4) + “summer season off just for you” (6) = 10 tokens.
- Ham: “meeting today” (2) + “call me later” (3) + “see you tomorrow” (3) = 8 tokens.

اکنون مقادیر خواسته‌شده را محاسبه می‌کنیم:

- کلمه‌ی just دو بار در ایمیل‌های spam ظاهر شده است (یک بار در هر ایمیل)، پس

$$\mathbb{P}(W = \text{just} | Y = \text{spam}) = \frac{2}{10} = \frac{1}{5}.$$

- کلمه‌ی call یک بار در ایمیل‌های ham ظاهر شده است، پس

$$\mathbb{P}(W = \text{call} | Y = \text{ham}) = \frac{1}{8}.$$

- کلمه‌ی offer هیچ‌گاه در ایمیل‌های ham دیده نشده است، پس چون از smoothing استفاده نمی‌کنیم

$$\mathbb{P}(W = \text{offer} | Y = \text{ham}) = \frac{0}{8} = 0.$$

(این همان مسئله‌ی فراوانی صفر است که با هموارسازی رفع می‌شود.)

- از میان 5 ایمیل، 2 ایمیل spam هستند، پس

$$\mathbb{P}(Y = \text{spam}) = \frac{2}{5}.$$

(آ) یک شبکه عصبی با حداکثر ۳ لایه مخفی طراحی کنید که خروجی آن برابر با تابع زیر باشد:

$$f(x_1, x_2, x_3) = \min(x_1, x_2^{x_3}).$$

توجه کنید که تنها ورودی‌های مجاز x_1, x_2, x_3 هستند و امکان افزودن ورودی کمکی به شبکه طراحی شده وجود ندارد. تابع فعال‌سازی هر نورون تنها روی ترکیب خطی ورودی‌های همان نورون اعمال می‌شود. برای توابع فعال‌سازی لایه‌های مخفی استفاده از تمامی توابع دلخواه مجاز است اما تابع فعال‌سازی لایه خروجی باید تابع همانی $f(x) = x$ باشد.

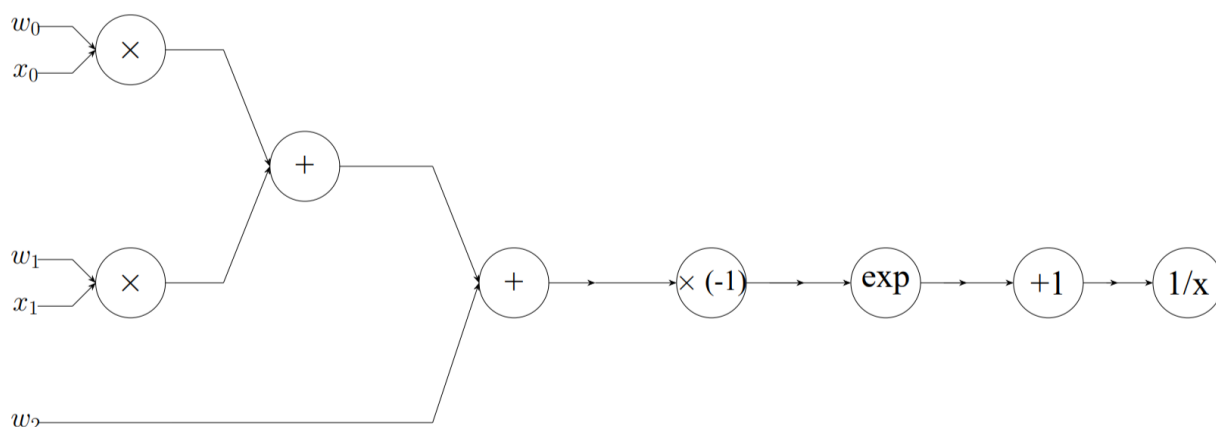
(ب) تابع زیر را در نظر بگیرید:

$$f(w, x) = \frac{1}{1 + e^{-(w \cdot x_0 + w_1 x_1 + w_2 x_2)}}$$

گراف محاسبات متناظر با این تابع در شکل زیر آمده است. با فرض ورودی‌های زیر:

$$w_0 = 2, \quad x_0 = -1, \quad w_1 = -3, \quad x_1 = -2, \quad w_2 = -3$$

ابتدا forward pass را انجام دهید و مقادیر محاسبه شده در هر مرحله را بنویسید. سپس با استفاده از این مقادیر محاسبات backward pass را انجام داده و مشتق خروجی نهایی را نسبت به خروجی هر رأس حساب کنید.



حل.

(آ) تابع خواسته شده را به دو بخش می‌شکنیم: نخست محاسبه‌ی توان $x_2^{x_3}$ و سپس گرفتن کمینه‌ی آن با x_1 . (فرض می‌کنیم $x_2 > 1$ تا $\ln(\ln x_2)$ و در نتیجه $x_2^{x_3}$ به‌خوبی تعریف شود.)
ایده‌ی محاسبه‌ی توان: با دو بار لگاریتم و دو بار نمای طبیعی، توان را تنها با ترکیب‌های خطی و توابع فعال‌سازی می‌سازیم:

$$x_2^{x_3} = \exp(x_3 \cdot \ln x_2) = \exp\left(\exp(\ln x_3 + \ln(\ln x_2))\right).$$

درستی رابطه روشن است، زیرا $\exp(\ln x_3 + \ln(\ln x_2)) = x_3 \cdot \ln x_2$ و در نتیجه نمای بیرونی همان $e^{x_3 \ln x_2} = x_2^{x_3}$ را می‌دهد.

ایده‌ی محاسبه‌ی کمینه: از این همانی زیر استفاده می‌کنیم:

$$\min(a, b) = \frac{a+b}{2} - \frac{|a-b|}{2}.$$

چون این رابطه ترکیبی خطی از دو کمیت $\frac{a+b}{2}$ و $\frac{|a-b|}{2}$ است، هرکدام را در یک نورون لایه سوم می‌سازیم و جمع نهایی را در لایه خروجی با تابع همانی انجام می‌دهیم.

ساختار شبکه (۳ لایه مخفی):

• لایه مخفی ۱:

$$f_1(x) = \ln(\ln x) \quad \text{inputs: } x_2 \text{ (weight 1).}$$

$$f_2(x) = \ln(x) \quad \text{inputs: } x_3 \text{ (weight 1).}$$

• لایه مخفی ۲:

$$f_3(x) = e^{e^x} \quad \text{inputs: } f_1 \text{ (weight 1), } f_2 \text{ (weight 1).}$$

$$f_3 = e^{e^{\ln(\ln x_2) + \ln x_3}} = x_2^{x_3}.$$

• لایه مخفی ۳:

$$f_4(x) = \frac{|x|}{2} \quad \text{inputs: } x_1 \text{ (weight -1), } f_3 \text{ (weight 1).}$$

$$f_5(x) = \frac{x}{2} \quad \text{inputs: } x_1 \text{ (weight 1), } f_3 \text{ (weight 1).}$$

$$f_4 = \frac{1}{2}|x_2^{x_3} - x_1|, \quad f_5 = \frac{1}{2}(x_1 + x_2^{x_3}).$$

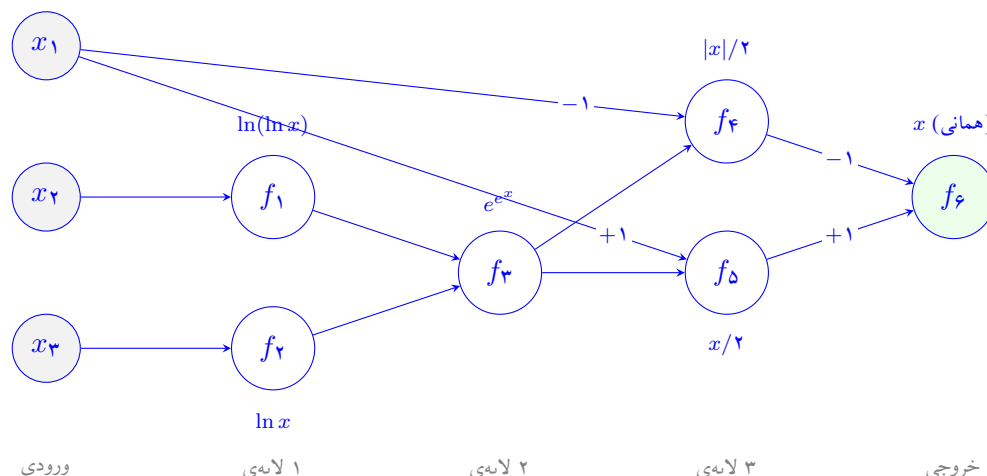
• لایه خروجی (فعال‌سازی همانی $f(x) = x$):

$$f_6(x) = x \quad \text{inputs: } f_4 \text{ (weight -1), } f_5 \text{ (weight 1).}$$

$$y = f_5 - f_4 = \frac{x_1 + x_2^{x_3}}{2} - \frac{|x_2^{x_3} - x_1|}{2} = \min(x_1, x_2^{x_3}).$$

خروجی شبکه دقیقاً برابر با $\min(x_1, x_2^{x_3})$ است و تنها از سه لایه مخفی و ورودی‌های مجاز x_1, x_2, x_3 استفاده شده است.

نمودار شبکه: شکل زیر ساختار شبکه‌ی طراحی شده را نشان می‌دهد (وزن‌هایی که روی یال نوشته نشده‌اند برابر ۱ هستند).



(ب) گراف محاسبات همان تابع سیگموئید روی $s = w_0x_0 + w_1x_1 + w_2x_2$ است. رئوس میانی را به ترتیب چنین نام‌گذاری می‌کنیم:

$$m_0 = w_0x_0, \quad m_1 = w_1x_1, \quad p = m_0 + m_1, \quad s = p + w_2, \quad n = -s, \quad e = e^n, \quad q = 1 + e, \quad f = \frac{1}{q}.$$

Forward pass: با جای‌گذاری $w_0 = 2, x_0 = -1, w_1 = -3, x_1 = -2, w_2 = -3$

$$\begin{aligned} m_{\cdot} &= w_{\cdot} x_{\cdot} = (2)(-1) = -2, & m_1 &= w_1 x_1 = (-3)(-2) = 6, \\ p &= m_{\cdot} + m_1 = -2 + 6 = 4, & s &= p + w_2 = 4 + (-3) = 1, \\ n &= -s = -1, & e &= e^{-1} \approx 0.3679, \\ q &= 1 + e \approx 1.3679, & f &= \frac{1}{q} \approx 0.7311. \end{aligned}$$

(همان‌طور که انتظار می‌رود، $f = \sigma(s) = \sigma(1) \approx 0.7311$)

Backward pass: مشتق خروجی نهایی f را نسبت به خروجی هر رأس، با قاعده‌ی زنجیره‌ای از انتها به ابتدا محاسبه می‌کنیم. بدیهی است $\frac{\partial f}{\partial f} = 1$.

$$\begin{aligned} \frac{\partial f}{\partial q} &= -\frac{1}{q^2} = -\frac{1}{(1.3679)^2} \approx -0.5345, \\ \frac{\partial f}{\partial e} &= \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial e} = (-0.5345)(1) \approx -0.5345, \\ \frac{\partial f}{\partial n} &= \frac{\partial f}{\partial e} \cdot \frac{\partial e}{\partial n} = (-0.5345)e^{-1} = (-0.5345)(0.3679) \approx -0.1966, \\ \frac{\partial f}{\partial s} &= \frac{\partial f}{\partial n} \cdot \frac{\partial n}{\partial s} = (-0.1966)(-1) \approx 0.1966, \\ \frac{\partial f}{\partial p} &= \frac{\partial f}{\partial s} \cdot 1 \approx 0.1966, \\ \frac{\partial f}{\partial w_2} &= \frac{\partial f}{\partial s} \cdot 1 \approx 0.1966, \\ \frac{\partial f}{\partial m_{\cdot}} &= \frac{\partial f}{\partial p} \cdot 1 \approx 0.1966, \\ \frac{\partial f}{\partial m_1} &= \frac{\partial f}{\partial p} \cdot 1 \approx 0.1966. \end{aligned}$$

مقدار $\frac{\partial f}{\partial s} \approx 0.1966$ با رابطه‌ی آشنای مشتق سیگموئید سازگار است:

$$\frac{\partial f}{\partial s} = \sigma(s)(1 - \sigma(s)) = (0.7311)(0.2689) \approx 0.1966.$$

مشتق نسبت به ورودی‌ها نیز از رئوس ضرب به دست می‌آید:

$$\begin{aligned} \frac{\partial f}{\partial w_{\cdot}} &= \frac{\partial f}{\partial m_{\cdot}} x_{\cdot} = (0.1966)(-1) \approx -0.1966, & \frac{\partial f}{\partial x_{\cdot}} &= \frac{\partial f}{\partial m_{\cdot}} w_{\cdot} = (0.1966)(2) \approx 0.3932, \\ \frac{\partial f}{\partial w_1} &= \frac{\partial f}{\partial m_1} x_1 = (0.1966)(-2) \approx -0.3932, & \frac{\partial f}{\partial x_1} &= \frac{\partial f}{\partial m_1} w_1 = (0.1966)(-3) \approx -0.5898. \end{aligned}$$

۵. (۱۵ نمره)

برای آموزش یک مدل رگرسیون لجستیک از تابع loss زیر استفاده می‌شود.

$$J(w) = \sum_{i=1}^n [-y^{(i)} \log(\sigma(w^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(w^T x^{(i)}))]$$

که در آن $\sigma(z) = \frac{1}{1+e^{-z}}$ ، و $y^{(i)} \in \{0, 1\}$ ، و $x^{(i)}$ بردار ویژگی نمونه i ام است.

- (آ) در مسائل طبقه‌بندی این تابع بیشتر ترجیح داده می‌شود یا تابع $SSE\ loss$ که از مربع اختلاف برچسب‌های واقعی و مقدار پیش‌بینی شده توسط مدل به صورت $(y - \sigma(w^T x))^2$ استفاده می‌کند؟ چرا؟
- (ب) مشتق گرادیان $\nabla_w J(w)$ را نسبت به بردار وزن w محاسبه کنید.
- (ج) قاعده به‌روزرسانی گرادیان کاهشی تصادفی (SGD) را برای یک نمونه $(x^{(i)}, y^{(i)})$ بنویسید.
- (د) اگر به جای مسئله دسته‌بندی دو کلاسه با مسئله دسته‌بندی سه کلاسه روبرو بودیم، تابع $loss$ و فرایند آموزش به چه شکل تغییر می‌کردند؟

حل.

- (آ) در مسائل طبقه‌بندی، تابع $loss$ آنتروپی متقاطع (همان $\log loss$ داده‌شده در صورت مسئله) بر $SSE\ loss$ ترجیح داده می‌شود. سه دلیل اصلی برای این ترجیح وجود دارد:
- ۱) **تحدب و بهینه‌سازی:** تابع آنتروپی متقاطع نسبت به بردار وزن w محدب است، بنابراین یک کمینه‌ی سراسری یکتا دارد و روش‌های گرادیانی به‌سادگی به آن همگرا می‌شوند. در مقابل، تابع

$$J_{SSE}(w) = \sum_{i=1}^n (y^{(i)} - \sigma(w^T x^{(i)}))^2$$

- به دلیل قرار گرفتن سیگموئید (که غیرخطی و اشباع‌شونده است) درون مربع، نسبت به w غیرمحدب است و می‌تواند کمینه‌های محلی متعدد داشته باشد که بهینه‌سازی را دشوار می‌کند.
- ۲) **مشکل محو شدن گرادیان:** گرادیان SSE برای هر نمونه به‌صورت

$$\frac{\partial}{\partial w} (y - \sigma(z))^2 = -2(y - \sigma(z)) \sigma'(z) x, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)),$$

است و شامل عامل $\sigma'(z)$ می‌شود. هنگامی که مدل با اطمینان اشتباه می‌کند (مثلاً $y = 1$ ولی $z \rightarrow -\infty$ و $\sigma(z) \approx 0$)، مقدار $\sigma'(z) \approx 0$ می‌شود و گرادیان به سمت صفر میل می‌کند؛ یعنی درست در جایی که بیشترین اصلاح لازم است، یادگیری بسیار کند یا متوقف می‌شود. اما همان‌طور که در بخش بعد می‌بینیم، گرادیان آنتروپی متقاطع فاقد این عامل است و در حالت اشتباه مطمئن، بزرگ باقی می‌ماند و یادگیری سریع انجام می‌شود.

- ۳) **تعبیر احتمالاتی:** کمینه کردن آنتروپی متقاطع دقیقاً معادل بیشینه‌سازی درست‌نمایی (Maximum Likelihood) تحت مدل برنولی $\mathbb{P}(y = 1 | x) = \sigma(w^T x)$ است، در حالی که SSE چنین تعبیر آماری طبیعی‌ای برای برچسب‌های گسسته ندارد و بیشتر برای مسائل رگرسیون با نوفه‌ی گاوسی مناسب است.
- (ب) یک نمونه‌ی دلخواه $(x^{(i)}, y^{(i)})$ را در نظر می‌گیریم و $z^{(i)} = w^T x^{(i)}$ می‌گذاریم. ابتدا مشتق $loss$ تک‌نمونه نسبت به z را محاسبه می‌کنیم:

$$J^{(i)} = -y^{(i)} \log \sigma(z^{(i)}) - (1 - y^{(i)}) \log (1 - \sigma(z^{(i)})).$$

با به‌کارگیری این همانی مشتق سیگموئید $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ داریم:

$$\frac{\partial J^{(i)}}{\partial z^{(i)}} = -y^{(i)} \frac{\sigma'(z^{(i)})}{\sigma(z^{(i)})} + (1 - y^{(i)}) \frac{\sigma'(z^{(i)})}{1 - \sigma(z^{(i)})}.$$

با جای‌گذاری $\sigma' = \sigma(1 - \sigma)$ و ساده‌سازی:

$$\frac{\partial J^{(i)}}{\partial z^{(i)}} = -y^{(i)} (1 - \sigma(z^{(i)})) + (1 - y^{(i)}) \sigma(z^{(i)}) = \sigma(z^{(i)}) - y^{(i)}.$$

اکنون با قاعده‌ی زنجیره‌ای و $\frac{\partial z^{(i)}}{\partial w} = x^{(i)}$ ، گرادیان کل روی n نمونه به‌دست می‌آید:

$$\nabla_w J(w) = \sum_{i=1}^n (\sigma(w^T x^{(i)}) - y^{(i)}) x^{(i)}$$

توجه کنید که برخلاف SSE، این گرادیان فاقد عامل $\sigma'(z)$ است و تنها به خطای پیش‌بینی $(\sigma(z^{(i)}) - y^{(i)})$ وابسته است.

(ج) در گرادیان کاهشی تصادفی (SGD)، در هر گام تنها از گرادیان یک نمونه‌ی $(x^{(i)}, y^{(i)})$ استفاده می‌کنیم و در خلاف جهت آن حرکت می‌کنیم. با نرخ یادگیری $\eta > 0$:

$$w \leftarrow w - \eta (\sigma(w^T x^{(i)}) - y^{(i)}) x^{(i)} = w + \eta (y^{(i)} - \sigma(w^T x^{(i)})) x^{(i)}$$

یعنی اگر پیش‌بینی $\sigma(w^T x^{(i)})$ کوچک‌تر از برچسب واقعی باشد، وزن‌ها در جهت $x^{(i)}$ افزایش می‌یابند و بالعکس؛ و اندازه‌ی گام متناسب با میزان خطاست.

(د) در حالت سه کلاسه (به‌طور کلی K کلاسه)، به‌جای یک بردار وزن، برای هر کلاس $k \in \{1, \dots, K\}$ یک بردار وزن w_k در نظر می‌گیریم و تابع سیگموئید را با تابع softmax جای‌گزین می‌کنیم که یک توزیع احتمال روی کلاس‌ها می‌سازد:

$$\mathbb{P}(y = k \mid x) = \frac{\exp(w_k^T x)}{\sum_{j=1}^K \exp(w_j^T x)} = p_k.$$

برچسب‌ها را به‌صورت بردار one-hot نمایش می‌دهیم؛ یعنی $y_k^{(i)} = 1$ اگر نمونه‌ی i به کلاس k تعلق داشته باشد و در غیر این صورت صفر. تابع loss نیز به آن‌روپی متقاطع دسته‌ای (categorical cross-entropy) تعمیم می‌یابد:

$$J(W) = - \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log(p_k^{(i)}).$$

(برای $K = 2$ این تابع دقیقاً به همان log loss دو کلاسه‌ی ابتدای مسئله تبدیل می‌شود.) فرایند آموزش ماهیتاً بدون تغییر می‌ماند و همچنان با SGD انجام می‌شود؛ تنها گرادیان برای هر بردار وزن به شکل زیر درمی‌آید که ساختار آشنای «پیش‌بینی منهای برچسب» را حفظ می‌کند:

$$\nabla_{w_k} J(W) = \sum_{i=1}^n (p_k^{(i)} - y_k^{(i)}) x^{(i)}, \quad w_k \leftarrow w_k - \eta (p_k^{(i)} - y_k^{(i)}) x^{(i)}.$$

۶. (۱۵ نمره) می‌خواهیم با استفاده از ویژگی‌های زیر پرسپترون برای تشخیص بیماری افراد آموزش دهیم. مدل پرسپترون به این صورت تعریف می‌شود:

$$\hat{y} = \text{sign}(w \cdot f + w_1 f_1 + w_2 f_2)$$

که در آن $f = 1$ بایاس ثابت و برچسب $y = +1$ نشان‌دهنده مبتلا بودن بیمار و $y = -1$ نشان‌دهنده سالم بودن او است. همچنین داده‌های آموزشی زیر را نیز در اختیار داریم:

بیمار	f_1 (قند)	f_2 (فشار)	y
A	۲	۱	+۱
B	-۱	۲	+۱
C	۱	-۲	-۱
D	-۲	-۱	-۱

بردار وزن اولیه را $(w_0, w_1, w_2) = (0, 0, 0)$ و نرخ یادگیری را $\eta = 1$ در نظر می‌گیریم. همچنین داده‌ها به ترتیب با شروع از A به الگوریتم داده می‌شوند.

- (آ) الگوریتم آموزش پرسپترون را یک مرحله روی تمامی داده‌ها اجرا کنید و در هر مرحله مقدار $\hat{y}$ و بردار وزن آپدیت شده را بنویسید.
- (ب) با وزن‌های به دست آمده در بخش (الف)، آیا تمام داده‌های آموزشی درست دسته‌بندی می‌شوند؟ اگر نه، آیا این لزوماً به معنای خطی جدا ناپذیر بودن داده‌هاست؟ توضیح دهید.
- (ج) نشان دهید داده‌ها خطی جداپذیر هستند یا نه. اگر بله، یک مرز تصمیم به صورت معادله ارائه دهید و نشان دهید که تمام نمونه‌ها را درست جدا می‌کند.

حل.

(آ) هر نمونه را با بردار ویژگی $f = (f_0, f_1, f_2) = (1, f_1, f_2)$ نشان می‌دهیم و پیش‌بینی را با قرارداد $\hat{y} = +1$ هرگاه $z = w^T f \geq 0$ و $\hat{y} = -1$ در غیر این صورت انجام می‌دهیم. هرگاه $\hat{y} \neq y$ باشد، قاعده‌ی به‌روزرسانی پرسپترون را با نرخ یادگیری $\eta = 1$ اعمال می‌کنیم:

$$w \leftarrow w + \eta y f = w + y f.$$

با شروع از $w = (0, 0, 0)$ و به‌ترتیب A, B, C, D :

• **نمونه‌ی A** : $A = (1, 2, 1), y = +1$

$$z = (0)(1) + (0)(2) + (0)(1) = 0 \geq 0 \Rightarrow \hat{y} = +1 = y.$$

پیش‌بینی درست است، پس به‌روزرسانی انجام نمی‌شود: $w = (0, 0, 0)$.

• **نمونه‌ی B** : $B = (1, -1, 2), y = +1$

$$z = (0)(1) + (0)(-1) + (0)(2) = 0 \geq 0 \Rightarrow \hat{y} = +1 = y.$$

پیش‌بینی درست است: $w = (0, 0, 0)$.

• **نمونه‌ی C** : $C = (1, 1, -2), y = -1$

$$z = 0 \geq 0 \Rightarrow \hat{y} = +1 \neq y = -1.$$

خطا رخ داده است، پس به‌روزرسانی می‌کنیم:

$$w \leftarrow (0, 0, 0) + (-1)(1, 1, -2) = (-1, -1, 2).$$

• **نمونه‌ی D** : $D = (1, -2, -1), y = -1$

$$z = (-1)(1) + (-1)(-2) + (2)(-1) = -1 + 2 - 2 = -1 < 0 \Rightarrow \hat{y} = -1 = y.$$

پیش‌بینی درست است: $w = (-1, -1, 2)$.

جمع‌بندی یک مرحله‌ی کامل روی داده‌ها:

sample	$z = w^T f$	$\hat{y}$	updated $w = (w_0, w_1, w_2)$
A	0	+1	(0, 0, 0)
B	0	+1	(0, 0, 0)
C	0	+1	(-1, -1, 2)
D	-1	-1	(-1, -1, 2)

پس بردار وزن نهایی پس از یک مرحله برابر است با $w = (-1, -1, 2)$.

(ب) با وزن‌های به‌دست‌آمده $w = (-1, -1, 2)$ ، پیش‌بینی هر نمونه را بررسی می‌کنیم:

sample	$z = w^T f$	$\hat{y}$	y
A	$-1 - 2 + 2 = -1$	-1	+1 ×
B	$-1 + 1 + 4 = 4$	+1	+1 ✓
C	$-1 - 1 - 4 = -6$	-1	-1 ✓
D	$-1 + 2 - 2 = -1$	-1	-1 ✓

بنابراین خیر؛ نمونه‌ی A همچنان نادرست دسته‌بندی می‌شود.

اما این لزوماً به معنای خطی جداناپذیر بودن داده‌ها نیست. الگوریتم پرسپترون تنها یک مرحله (یک عبور) روی داده‌ها اجرا شده است، در حالی که همگرایی معمولاً به چند عبور نیاز دارد؛ به‌ویژه به‌روزرسانی روی بردار وزن را چنان جابه‌جا کرده که اکنون A در سمت اشتباه مرز قرار گرفته است. تا زمانی که داده‌ها خطی جداناپذیر باشند، قضیه‌ی همگرایی پرسپترون تضمین می‌کند که با ادامه‌ی تکرارها الگوریتم پس از تعداد متناهی گام به یک جداکننده‌ی کامل می‌رسد. پس وجود خطا پس از یک عبور، شاهده‌ی بر جداناپذیری نیست.

(ج) داده‌ها خطی جداپذیر هستند. کافی است یک مرز تصمیم ارائه دهیم که همه‌ی نمونه‌ها را درست جدا کند. مرز

$$f_1 + f_2 = 0$$

را در نظر می‌گیریم که متناظر با بردار وزن $(w_0, w_1, w_2) = (0, 1, 1)$ و در نتیجه $z = f_1 + f_2$ است. مقدار z و پیش‌بینی هر نمونه:

sample	$z = f_1 + f_2$	$\hat{y} = \text{sign}(z)$	y
A	$2 + 1 = 3$	+1	+1 ✓
B	$-1 + 2 = 1$	+1	+1 ✓
C	$1 - 2 = -1$	-1	-1 ✓
D	$-2 - 1 = -3$	-1	-1 ✓

هر چهار نمونه به‌درستی دسته‌بندی می‌شوند: نمونه‌های مثبت (A, B) همگی $z > 0$ و نمونه‌های منفی (C, D) همگی $z < 0$ دارند. بنابراین خط $f_1 + f_2 = 0$ یک مرز تصمیم معتبر است و داده‌ها خطی جداناپذیرند.

(به‌طور شهودی، این ساختار طبیعی است: نمونه‌ها به‌صورت متقارن حول مبدأ قرار دارند، چون $C = -B$ و $D = -A$ و برچسب‌های آن‌ها قرینه است؛ از این رو خطی گذرنده از مبدأ می‌تواند دو کلاس را از هم جدا کند.)

۷. (۸ نمره)

(آ) با استفاده از شرط نابرابری Jensen، محدب بودن یا نبودن توابع زیر را بر روی دامنه‌های داده شده بررسی کنید:

- $f(x) = \sqrt{x}$ بر روی $[0, \infty)$
 - $f(x) = x^2$ بر روی مجموعه کل اعداد حقیقی
- (ب) با بررسی مشتق دوم، محدب بودن یا نبودن توابع زیر را بر روی دامنه‌های داده شده بررسی کنید:
- $f(x) = -\log(x)$ بر روی $(0, \infty)$
 - $f(x) = x^4 - 2x^2$ بر روی مجموعه کل اعداد حقیقی

حل.

(آ) تابع f روی یک بازه‌ی محدب محدب است هرگاه برای هر دو نقطه‌ی x, y از دامنه و هر $\lambda \in [0, 1]$ نابرابری Jensen برقرار باشد:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

اگر همواره جهت نابرابری برعکس باشد تابع مقعر است؛ و برای اثبات محدب نبودن کافی است تنها یک مثال نقض بیابیم که در آن نابرابری بالا نقض شود.

- $f(x) = \sqrt{x}$ روی $[0, \infty)$: ادعا می‌کنیم این تابع محدب نیست (بلکه مقعر است). برای نقض، نقاط $x = 0$ و $y = 4$ و $\lambda = \frac{1}{2}$ را برمی‌گزینیم. طرف چپ نابرابری Jensen:

$$f\left(\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 4\right) = f(2) = \sqrt{2} \approx 1.414,$$

و طرف راست آن:

$$\frac{1}{2}f(0) + \frac{1}{2}f(4) = \frac{1}{2}(0) + \frac{1}{2}(2) = 1.$$

- چون $f(2) = 1.414 > 1$ ، شرط محدب بودن نقض می‌شود. پس $\sqrt{x}$ روی $[0, \infty)$ محدب نیست. (در واقع نمودار تابع بالای وترِ واصل دو نقطه قرار می‌گیرد، که نشانه‌ی مقعر بودن است.)
- $f(x) = x^2$ روی $\mathbb{R}$: ادعا می‌کنیم این تابع محدب است. تفاضل دو طرف نابرابری Jensen را برای x, y و $\lambda \in [0, 1]$ دلخواه محاسبه می‌کنیم:

$$\lambda f(x) + (1 - \lambda)f(y) - f(\lambda x + (1 - \lambda)y) = \lambda x^2 + (1 - \lambda)y^2 - (\lambda x + (1 - \lambda)y)^2.$$

با بسط جمله‌ی آخر و ساده‌سازی به دست می‌آید:

$$= \lambda x^2 + (1 - \lambda)y^2 - \lambda^2 x^2 - 2\lambda(1 - \lambda)xy - (1 - \lambda)^2 y^2 = \lambda(1 - \lambda)(x - y)^2.$$

چون $\lambda \in [0, 1]$ است، هر دو عامل λ و $1 - \lambda$ نامنفی‌اند و $(x - y)^2 \geq 0$ ؛ بنابراین کل عبارت نامنفی است:

$$\lambda f(x) + (1 - \lambda)f(y) - f(\lambda x + (1 - \lambda)y) = \lambda(1 - \lambda)(x - y)^2 \geq 0.$$

پس نابرابری Jensen برای همه‌ی x, y, λ برقرار است و x^2 روی کل $\mathbb{R}$ محدب است.

- (ب) یادآوری می‌کنیم که یک تابع دو بار مشتق‌پذیر روی یک بازه محدب است اگر و تنها اگر مشتق دومش روی سراسر آن بازه نامنفی باشد: $f''(x) \geq 0$.

- $f(x) = -\log(x)$ روی $(0, \infty)$: مشتق‌های اول و دوم را می‌گیریم:

$$f'(x) = -\frac{1}{x}, \quad f''(x) = \frac{1}{x^2}.$$

برای هر $x > 0$ داریم $f''(x) = \frac{1}{x^2} > 0$. چون مشتق دوم روی سراسر دامنه‌ی $(0, \infty)$ اکیداً مثبت است، تابع $-\log(x)$ روی این دامنه محدب (بلکه اکیداً محدب) است.

• $f(x) = x^4 - 2x^2$ روی $\mathbb{R}$: مشتق‌های اول و دوم:

$$f'(x) = 4x^3 - 4x, \quad f''(x) = 12x^2 - 4 = 4(3x^2 - 1).$$

علامت f'' ثابت نیست: داریم $f''(x) \geq 0$ تنها هنگامی که $x^2 \geq \frac{1}{3}$ ، یعنی $|x| \geq \frac{1}{\sqrt{3}}$ ؛ ولی برای $|x| < \frac{1}{\sqrt{3}}$ مقدار مشتق دوم منفی است. برای نمونه در $x = 0$ داریم:

$$f''(0) = 12(0)^2 - 4 = -4 < 0.$$

پس چون f'' روی کل $\mathbb{R}$ نامنفی نیست، تابع $x^4 - 2x^2$ روی مجموعه‌ی کل اعداد حقیقی محدب نیست. (این تابع تنها روی بازه‌های $(-\infty, -\frac{1}{\sqrt{3}}]$ و $[\frac{1}{\sqrt{3}}, \infty)$ به‌طور موضعی محدب است و در بازه‌ی میانی مقعر است.)

۸. (۱۰ نمره) امتیازی.

(آ) یک نقشه‌ی ویژگی ورودی (Input Feature Map) و یک فیلتر (Kernel) به‌صورت زیر در اختیار داریم:

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 1 \\ 0 & 1 & 3 & 1 \\ 2 & 3 & 1 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

اطلاعات کانولوشن به‌شکل زیر هستند:

- اندازه‌ی فیلتر: 2×2
- مقدار padding: $p = 1$
- مقدار stride: $s = 1$
- بایاس: صفر

عملیات کانولوشن را به‌طور کامل انجام داده و نقشه‌ی ویژگی خروجی را به دست آورید.

(ب) فرض کنید یک دنباله‌ی ورودی متنی شامل دو توکن باشد. در یک لایه‌ی Attention از مدل ترنسفورمر، بردارهای Query، Key و Value برای این دو توکن به‌صورت زیر به دست آمده‌اند:

$$Q = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

که در آن:

- هر سطر متناظر با یک توکن است.
- بُعد بردارهای Query و Key و Value برابر $d = 2$ است.
- مکانیزم توجه از نوع Dot-Product است.
- i. ماتریس امتیازهای توجه (Attention Score) را محاسبه کنید.
- ii. امتیازهای نرمالایز شده‌ی توجه را با اعمال تابع softmax روی هر سطر از ماتریس امتیازها به‌دست آورید (استفاده از ماشین حساب مجاز است).
- iii. خروجی نهایی مکانیزم Attention را محاسبه کنید.

حل.

(آ) نخست با $p = 1$ دور تا دور ورودی X یک ردیف صفر می‌افزاییم تا ماتریس 6×6 زیر حاصل شود:

$$X_{\text{pad}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & 3 & 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 3 & 1 & 0 \\ 0 & 2 & 3 & 1 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

اندازه‌ی خروجی از رابطه‌ی زیر به دست می‌آید:

$$O = \frac{N - F + 2p}{s} + 1 = \frac{4 - 2 + 2 \cdot 1}{1} + 1 = 5,$$

پس نقشه‌ی ویژگی خروجی 5×5 است. با فیلتر

$$K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix},$$

هر درایه‌ی خروجی برابر است با مجموع حاصل ضرب درایه‌های فیلتر در پنجره‌ی متناظر آن روی X_{pad} . با نام‌گذاری درایه‌های پنجره به صورت $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ ، مقدار خروجی برابر است با $1 \cdot a + (-1) \cdot b + 0 \cdot c + 1 \cdot d = a - b + d$.

برای نمونه، درایه‌ی گوشه‌ی بالا-چپ خروجی از پنجره‌ی $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ حاصل می‌شود:

$$O_{1,1} = 0 - 0 + 1 = 1,$$

و به‌عنوان نمونه‌ای دیگر، درایه‌ی مرکزی از پنجره‌ی $\begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}$:

$$O_{3,3} = 1 - 2 + 3 = 2.$$

با تکرار این محاسبه برای هر ۲۵ موقعیت پنجره، نقشه‌ی ویژگی خروجی نهایی به دست می‌آید:

$$O = \begin{bmatrix} 1 & 2 & 0 & 1 & 0 \\ 2 & 0 & 4 & 0 & 1 \\ -3 & 3 & 2 & 2 & 1 \\ 2 & 2 & -1 & 4 & 1 \\ -2 & -1 & 2 & -1 & 2 \end{bmatrix}$$

(ب) هر سطر از ماتریس‌های Q, K, V متناظر یک توکن است و بُعد بردارها $d = 2$ می‌باشد.

i. **ماتریس امتیازهای توجه.** در توجه از نوع Dot-Product، ماتریس امتیازها از ضرب داخلی هر

بردار Query در هر بردار Key، یعنی $S = QK^T$ ، به دست می‌آید. با

$$Q = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad K^T = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix},$$

داریم:

$$S = QK^T = \begin{bmatrix} (1)(0) + (2)(1) & (1)(1) + (2)(1) \\ (0)(0) + (1)(1) & (0)(1) + (1)(1) \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix}.$$

درایه‌ی S_{ij} میزان شباهت (توجه) توکن i به توکن j را نشان می‌دهد.

ii. **نرمال سازی با softmax.** پیش از اعمال softmax، مطابق فرمول توجه مقیاس دار (Scaled Dot-Product Attention)، امتیازها را بر $\sqrt{d}$ تقسیم می‌کنیم تا از بزرگ شدن بیش از حد ضرب‌های داخلی و اشباع گرادین softmax جلوگیری شود. با $d = 2$ داریم $\sqrt{d} = \sqrt{2} \approx 1.414$ ، پس:

$$\frac{S}{\sqrt{d}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix} \approx \begin{bmatrix} 1/414 & 2/121 \\ 0/707 & 0/707 \end{bmatrix}.$$

اکنون تابع softmax را روی هر سطر جداگانه اعمال می‌کنیم تا هر سطر به یک توزیع احتمال (با مجموع یک) تبدیل شود. برای سطر اول:

$$\text{softmax}(1/414, 2/121) = \left(\frac{e^{1/414}}{e^{1/414} + e^{2/121}}, \frac{e^{2/121}}{e^{1/414} + e^{2/121}} \right) \approx (0/330, 0/670),$$

و برای سطر دوم، چون هر دو امتیاز برابرند:

$$\text{softmax}(0/707, 0/707) = \left(\frac{e^{0/707}}{e^{0/707} + e^{0/707}}, \frac{e^{0/707}}{e^{0/707} + e^{0/707}} \right) = (0/5, 0/5).$$

بنابراین ماتریس وزن‌های نرمال‌شده‌ی توجه چنین است:

$$A = \begin{bmatrix} 0/330 & 0/670 \\ 0/5 & 0/5 \end{bmatrix}.$$

iii. **خروجی نهایی مکانیزم Attention.** خروجی از ضرب ماتریس وزن‌های توجه در ماتریس Value، یعنی $\text{Out} = AV$ ، به دست می‌آید. با

$$V = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix},$$

سطر اول خروجی (ترکیب وزن‌دار مقادیر برای توکن اول):

$$\text{Out}_1 = 0/330 (2, 1) + 0/670 (1, 3) = (0/660 + 0/670, 0/330 + 2/010) \approx (1/330, 2/340),$$

و سطر دوم:

$$\text{Out}_2 = 0/5 (2, 1) + 0/5 (1, 3) = (1/5, 2/0).$$

پس خروجی نهایی لایه‌ی Attention برابر است با:

$$\text{Out} = AV \approx \begin{bmatrix} 1/330 & 2/340 \\ 1/5 & 2/0 \end{bmatrix}.$$

هر سطر، بازنمایی به‌روزشده‌ی توکن متناظر است که به‌صورت میانگین وزن‌دار بردارهای Value (با وزن‌های حاصل از توجه) ساخته شده است.